

INFORMATION RETRIEVAL

Scoring, term weighting & the vector space model

Corso di Laurea Magistrale in Informatica

Università di Roma Tor Vergata

Prof. Giorgio Gambosi

a.a. 2021-2022

Derived from slides originally produced by C. Manning and by H. Schütze



- ⊙ Boolean queries.
 - Documents either match or don't.
- ⊙ Good for expert users with precise understanding of their needs and of the collection.
- ⊙ Also good for applications: Applications can easily consume 1000s of results.
- ⊙ Not good for the majority of users
 - Most users are not capable of writing Boolean queries ...
 - ...or they are, but they think it's too much work.
 - Most users don't want to wade through 1000s of results.
 - This is particularly true of web search.

Problem with Boolean search: Feast or famine

- ⊙ Boolean queries often result in either too few (=0) or too many (1000s) results.
- ⊙ Query 1 (boolean conjunction): “world climate crisis”
 - → 200,000 hits – **feast**
- ⊙ Query 2 (boolean conjunction): “world climate crisis merkel”
 - → 0 hits – **famine**
- ⊙ In Boolean retrieval, it takes a lot of skill to come up with a query that produces a manageable number of hits.
 - AND gives too few; OR gives too many
- ⊙ Suggested solution:
 - Rank documents by goodness – a sort of clever “soft AND”

Feast or famine: No problem in ranked retrieval

With ranking, large result sets are not an issue.

- ⦿ Just show the top 10 results
- ⦿ Doesn't overwhelm the user
- ⦿ Premise: the ranking algorithm works, that is, **more relevant results are ranked higher than less relevant results.**

Scoring as the basis of ranked retrieval

- ⊙ How can we accomplish a **relevance ranking** of the documents with respect to a query?
- ⊙ Assign a score to each query-document pair, say in $[0, 1]$.
- ⊙ This score measures how well document and query “match”.
- ⊙ Sort documents according to scores

How do we compute the score of a query-document pair?

- ⦿ If no query term occurs in the document: score should be 0.
- ⦿ The more frequent a query term in the document, the higher the score
- ⦿ The more query terms occur in the document, the higher the score

Jaccard coefficient

A commonly used measure of overlap of two sets

- ⊙ Let A and B be two sets
- ⊙ Jaccard coefficient:

$$\text{JACCARD}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

$(A \neq \emptyset \text{ or } B \neq \emptyset)$

- ⊙ $\text{JACCARD}(A, A) = 1$
- ⊙ $\text{JACCARD}(A, B) = 0$ if $A \cap B = \emptyset$
- ⊙ A and B don't have to be the same size.
- ⊙ Always assigns a number between 0 and 1.

Jaccard coefficient: Example

- ⊙ What is the query-document match score that the Jaccard coefficient computes for:
 - Query: “ides of March”
 - Document “Caesar died in March”
 - $\text{JACCARD}(q, d) = 1/6$

What's wrong with Jaccard?

- ⊙ It doesn't consider term frequency (how many occurrences a term has).
- ⊙ Rare terms are more informative than frequent terms. Jaccard does not consider this information.
- ⊙ Usually, $\frac{|A \cap B|}{\sqrt{|A \cup B|}}$ (cosine) seems better than $\frac{|A \cap B|}{|A \cup B|}$ (Jaccard) for length normalization.

Query-document matching scores

- ⊙ We need a way of assigning a score to a query/document pair
- ⊙ Let's start with a one-term query
- ⊙ If the query term does not occur in the document: score should be 0
- ⊙ The more frequent the query term in the document, the higher the score (should be)

Binary incidence matrix

Consider the occurrence of a term in a document:

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Each document is represented as a **binary vector** $\in \{0, 1\}^{|V|}$.

Count matrix

Consider the number of occurrences of a term in a document:

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	157	73	0	0	0	1	
Brutus	4	157	0	2	0	0	
Caesar	232	227	0	2	1	0	
Calpurnia	0	10	0	0	0	0	
Cleopatra	57	0	0	0	0	0	
mercy	2	0	3	8	5	8	
worser	2	0	1	1	1	5	
...							

Each document is now represented as a **count vector** $\in \mathbb{N}^{|V|}$.

Bag of words model

- ⊙ We do not consider the **order** of words in a document.
- ⊙ *John is quicker than Mary* and *Mary is quicker than John* are represented the same way.
- ⊙ This is called a **bag of words model**.
- ⊙ Information loss, but simplification of the problem: the positional index was able to distinguish these two documents.

Term frequency tf

- ⊙ The term frequency $tf_{t,d}$ of term t in document d is defined as the number of times that t occurs in d .
- ⊙ We want to use tf when computing query-document match scores.
- ⊙ But how?
- ⊙ Raw term frequency is not what we want because:
 - A document with $tf = 10$ occurrences of the term is more relevant than a document with $tf = 1$ occurrence of the term.
 - But not 10 times more relevant.
- ⊙ Relevance does not increase proportionally with term frequency.

Instead of raw frequency: Log frequency weighting

- ⊙ The log frequency weight of term t in d is defined as

$$w_{t,d} = \begin{cases} 1 + \log_{10} \text{tf}_{t,d} & \text{if } \text{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$$

- ⊙ $\text{tf}_{t,d} \rightarrow w_{t,d}$:
 $0 \rightarrow 0, 1 \rightarrow 1, 2 \rightarrow 1.3, 10 \rightarrow 2, 1000 \rightarrow 4$, etc.
- ⊙ Score for a document-query pair: sum over terms t in both q and d :
 $\text{tf-matching-score}(q, d) = \sum_{t \in q \cap d} (1 + \log \text{tf}_{t,d})$
- ⊙ The score is 0 if none of the query terms is present in the document.

Compute the Jaccard matching score and the tf matching score for the following query-document pairs.

- ⊙ q: [information on cars] d: “all you’ve ever wanted to know about cars”
- ⊙ q: [information on cars] d: “information on trucks, information on planes, information on trains”
- ⊙ q: [red cars and red trucks] d: “cops stop red cars more often”

Frequency in document vs. frequency in collection

- ⊙ In addition, to term frequency (the frequency of the term in the document) ...
- ⊙ ...we also want to use the frequency of the term **in the collection** for weighting and ranking.

Desired weight for rare terms

- ⊙ Rare terms are more informative than frequent terms.
- ⊙ Consider a term in the query that is **rare** in the collection (e.g., **Phenethylamine**).
- ⊙ A document containing this term is very likely to be relevant.
- ⊙ We want **high weights for rare terms** like **Phenethylamine**.

Desired weight for frequent terms

- ⊙ Frequent terms are less informative than rare terms.
- ⊙ Consider a term in the query that is **frequent** in the collection (e.g., **good**, **increase**, **line**).
- ⊙ A document containing this term is more likely to be relevant than a document that doesn't
- ⊙ But words like **good**, **increase** and **line** are not sure indicators of relevance.
- ⊙ As a consequence, for frequent terms like **good**, **increase**, and **line**, we want positive weights,
- ⊙ but **lower weights** than for rare terms.

- ⊙ We want **high weights for rare terms** like **Phenethylamine**.
- ⊙ We want **low (positive) weights for frequent words** like **good**, **increase**, and **line**.
- ⊙ We will use **document frequency** to factor this into computing the matching score.
- ⊙ The document frequency is the number of documents in the collection that the term occurs in.

- ⊙ df_t is the document frequency, the number of documents that t occurs in.
- ⊙ df_t is an inverse measure of the **informativeness** of term t .
- ⊙ We define the **idf weight** of term t as follows:

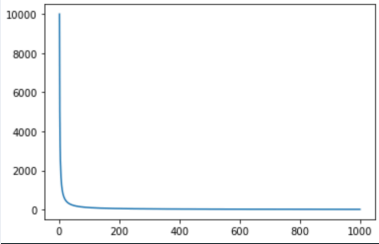
$$idf_t = \log_{10} \frac{N}{df_t}$$

(N is the number of documents in the collection.)

- ⊙ idf_t is a measure of the **informativeness** of the term.
- ⊙ $\log \frac{N}{df_t}$ instead of $\frac{N}{df_t}$ to “dampen” the effect of idf
- ⊙ Note that we use the log transformation for both term frequency and document frequency.

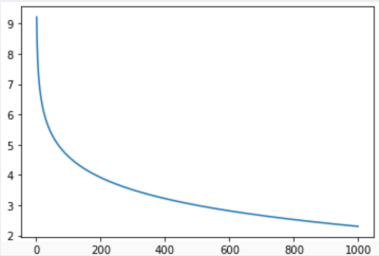
idf weight

⊙



$$\frac{N}{df_t}$$

⊙



$$\log \frac{N}{df_t}$$

Examples for idf

Compute idf_t using the formula: $\text{idf}_t = \log_{10} \frac{1,000,000}{\text{df}_t}$

term	df_t	idf_t
calpurnia	1	
animal	100	
sunday	1000	
fly	10,000	
under	100,000	
the	1,000,000	

Examples for idf

Compute idf_t using the formula: $\text{idf}_t = \log_{10} \frac{1,000,000}{\text{df}_t}$

term	df_t	idf_t
calpurnia	1	6
animal	100	4
sunday	1000	3
fly	10,000	2
under	100,000	1
the	1,000,000	0

Effect of idf on ranking

- ⊙ idf affects the ranking of documents for queries with at least two terms.
- ⊙ For example, in the query “arachnocentric line”, idf weighting **increases** the relative weight of **arachnocentric** and **decreases** the relative weight of **line**.
- ⊙ idf has **little effect** on ranking for one-term queries.

Collection frequency vs. Document frequency

word	collection frequency	document frequency
insurance	10440	3997
try	10422	8760

- ⊙ Collection frequency of t : **number of tokens of t in the collection**
- ⊙ Document frequency of t : **number of documents t occurs in**
- ⊙ Which word is a better search term (and should get a higher weight)?
- ⊙ This example suggests that df (and idf) is better for weighting than cf (and “icf”).

- ⊙ The tf-idf weight of a term is the product of its tf weight and its idf weight.

- ⊙

$$w_{t,d} = (1 + \log \text{tf}_{t,d}) \cdot \log \frac{N}{\text{df}_t}$$

- ⊙ tf-weight
- ⊙ idf-weight
- ⊙ Best known weighting scheme in information retrieval
- ⊙ Alternative names: tf.idf, tf x idf

- ⊙ Assign a tf-idf weight for each term t in each document d : $w_{t,d} = (1 + \log \text{tf}_{t,d}) \cdot \log \frac{N}{\text{df}_t}$
- ⊙ The tf-idf weight ...
 - ...increases with the number of occurrences within a document. (term frequency)
 - ...increases with the rarity of the term in the collection. (inverse document frequency)

Exercise: Term, collection and document frequency

Quantity	Symbol	Definition
term frequency	$tf_{t,d}$	number of occurrences of t in d
document frequency	df_t	number of documents in the collection that t occurs in
collection frequency	cf_t	total number of occurrences of t in the collection

- ⊙ Relationship between df and cf ?
- ⊙ Relationship between tf and cf ?
- ⊙ Relationship between tf and df ?

Binary incidence matrix

Consider the occurrence of a term in a document:

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Each document is represented as a **binary vector** $\in \{0, 1\}^{|V|}$.

Count matrix

Consider the number of occurrences of a term in a document:

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	157	73	0	0	0	1	
Brutus	4	157	0	2	0	0	
Caesar	232	227	0	2	1	0	
Calpurnia	0	10	0	0	0	0	
Cleopatra	57	0	0	0	0	0	
mercy	2	0	3	8	5	8	
worser	2	0	1	1	1	5	
...							

Each document is now represented as a **count vector** $\in \mathbb{N}^{|V|}$.

Binary → count → weight matrix

Consider the tf-idf score of a term in a document

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	5.25	3.18	0.0	0.0	0.0	0.35	
Brutus	1.21	6.10	0.0	1.0	0.0	0.0	
Caesar	8.59	2.54	0.0	1.51	0.25	0.0	
Calpurnia	0.0	1.54	0.0	0.0	0.0	0.0	
Cleopatra	2.85	0.0	0.0	0.0	0.0	0.0	
mercy	1.51	0.0	1.90	0.12	5.25	0.88	
worser	1.37	0.0	0.11	4.15	0.25	1.95	
...							

Each document is now represented as a **real-valued vector** of tf-idf weights $\in \mathbb{R}^{|V|}$.

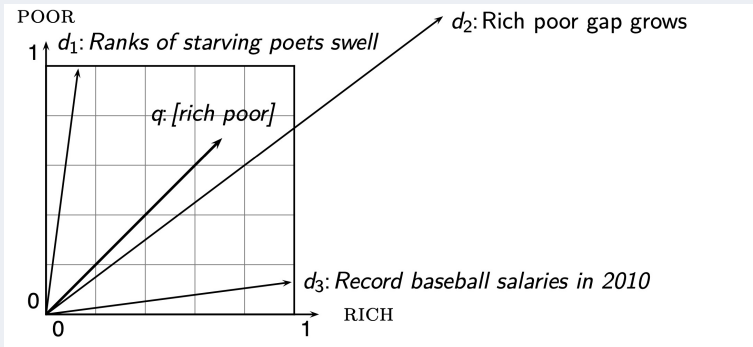
- ⊙ Each document is now represented as a real-valued vector of tf-idf weights $\in \mathbb{R}^{|V|}$.
- ⊙ So we have a $|V|$ -dimensional real-valued vector space.
- ⊙ Terms are **axes** of the space.
- ⊙ Documents are **points** or **vectors** in this space.
- ⊙ Very high-dimensional: tens of millions of dimensions when you apply this to web search engines
- ⊙ Each vector is very sparse - most entries are zero.

- ⊙ Key idea 1: do the same for queries: represent them as vectors in the high-dimensional space
- ⊙ Key idea 2: Rank documents according to their proximity to the query
- ⊙ proximity = similarity $\approx$ negative distance
- ⊙ Rank documents in inverse order wrt the **distance** of its vector from the query vector
- ⊙ How to define a **distance** between vectors of terms?

How do we formalize vector space similarity?

- ⊙ First approach: distance of vectors = distance between their endpoints
- ⊙ For example, euclidean distance
- ⊙ Endpoint distance is a bad idea: it is heavily affected by vector lengths
- ⊙ It may be **large** for vectors **of different lengths**

Why distance is a bad idea



The Euclidean distance of $\vec{q}$ and $\vec{d}_2$ is large although the distribution of terms in the query q and the distribution of terms in the document d_2 are very similar.

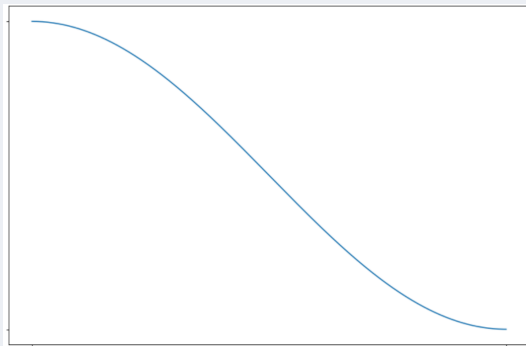
Why distance is a bad idea

- ⊙ Thought experiment: take a document d and append it to itself. Call this document d' . d' is twice as long as d .
- ⊙ “Semantically” d and d' have the same content.
- ⊙ The angle between the two documents is 0, corresponding to maximal similarity
- ⊙ The Euclidean distance between the two documents can be quite large.

Better approach: rank documents according to **angle** with query

Cosine function

The cosine function is monotonically decreasing in $[0, 2\pi]$



- ⊙ The following two notions are equivalent.
 - Rank documents according to the **angle** between query and document in decreasing order
 - Rank documents according to **cosine**(query,document) in increasing order
- ⊙ Cosine is a monotonically decreasing function of the angle for the interval $[0^\circ, 180^\circ]$

Cosine distance and length normalization

- ⊙ A vector can be normalized by dividing each of its components by its length (norm)
- ⊙ here we use the L_2 (euclidean) norm: $\|x\|_2 = \sqrt{\sum_i x_i^2}$
- ⊙ This maps vectors onto the unit sphere, since after normalization: $\|x\|_2 = \sqrt{\sum_i x_i^2} = 1$
- ⊙ As a result, longer documents and shorter documents have weights of the same order of magnitude.
- ⊙ Effect on the two documents d and d' (d appended to itself) from earlier slide: they have **identical vectors** after length normalization.

Cosine for normalized vectors

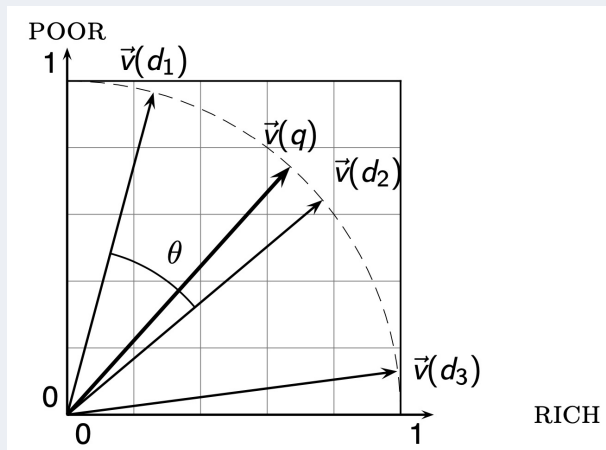
- ⊙ For normalized vectors, the cosine is equivalent to the dot (or scalar) product.
- ⊙ $\cos(\vec{q}, \vec{d}) = \vec{q} \cdot \vec{d} = \sum_i q_i \cdot d_i$
 - (if $\vec{q}$ and $\vec{d}$ are length-normalized).
- ⊙ this result in an approach to compute cosine similarity:
 - normalize vectors
 - sum of products for all components different from 0 in both vectors (terms appearing in both documents or in both document and query)

Cosine similarity between query and document

$$\cos(\vec{q}, \vec{d}) = \text{sim}(\vec{q}, \vec{d}) = \frac{\vec{q}}{|\vec{q}|} \cdot \frac{\vec{d}}{|\vec{d}|} = \frac{\vec{q} \cdot \vec{d}}{|\vec{q}| |\vec{d}|} = \frac{\sum_{i=1}^{|V|} q_i d_i}{\sqrt{\sum_{i=1}^{|V|} q_i^2} \sqrt{\sum_{i=1}^{|V|} d_i^2}}$$

- ⊙ q_i is the tf-idf weight of term i in the query.
- ⊙ d_i is the tf-idf weight of term i in the document.
- ⊙ $|\vec{q}|$ and $|\vec{d}|$ are the lengths of $\vec{q}$ and $\vec{d}$.
- ⊙ This is the **cosine similarity** of $\vec{q}$ and $\vec{d}$ or, equivalently, the cosine of the angle between $\vec{q}$ and $\vec{d}$.

Cosine similarity illustrated



How similar are these novels?

SaS: Sense and Sensibility
PaP: Pride and Prejudice
WH: Wuthering Heights

term frequencies (counts)

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

Cosine: Example

term frequencies (counts)

term	SaS	PaP	WH
affection	115	58	20
jealous	10	7	11
gossip	2	0	6
wuthering	0	0	38

log frequency weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.0	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

(To simplify this example, we don't do idf weighting.)

Cosine: Example

log frequency weighting

term	SaS	PaP	WH
affection	3.06	2.76	2.30
jealous	2.0	1.85	2.04
gossip	1.30	0	1.78
wuthering	0	0	2.58

log frequency weighting
& cosine normalization

term	SaS	PaP	WH
affection	0.789	0.832	0.524
jealous	0.515	0.555	0.465
gossip	0.335	0.0	0.405
wuthering	0.0	0.0	0.588

- ⊙ $\cos(\text{SaS}, \text{PaP}) \approx 0.789 * 0.832 + 0.515 * 0.555 + 0.335 * 0.0 + 0.0 * 0.0 \approx 0.94.$
- ⊙ $\cos(\text{SaS}, \text{WH}) \approx 0.79$
- ⊙ $\cos(\text{PaP}, \text{WH}) \approx 0.69$
- ⊙ Why do we have $\cos(\text{SaS}, \text{PaP}) > \cos(\text{SAS}, \text{WH})?$

Computing the cosine score

COSINESCORE(q)

```
1  float Scores[ $N$ ] = 0
2  float Length[ $N$ ]
3  for each query term  $t$ 
4  do calculate  $w_{t,q}$  and fetch postings list for  $t$ 
5      for each pair( $d, tf_{t,d}$ ) in postings list
6      do Scores[ $d$ ]+ =  $w_{t,d} \times w_{t,q}$ 
7  Read the array Length
8  for each  $d$ 
9  do Scores[ $d$ ] = Scores[ $d$ ]/Length[ $d$ ]
10 return Top  $K$  components of Scores[]
```

Computing the cosine score

- ⊙ The previous algorithm scores term-at-a-time (TAAT)
- ⊙ Algorithm can be adapted to scoring document-at-a-time (DAAT)

Storing $w_{t,d}$ in each posting could be expensive

- ⊙ ...because we'd have to store a floating point number
- ⊙ For tf-idf scoring, it suffices to store tft,d in the posting and $idft$ in the head of the postings list

Extracting the top K items can be done with a priority queue (e.g., a heap)

Variants of tf weighting

natural	$TF_{total}(t, d)$	$n(t, d)$
boolean	$TF_{bool}(t, d)$	$\begin{cases} 1 & \text{if } n(t, d) > 0 \\ 0 & \text{otherwise} \end{cases}$
sum	$TF_{sum}(t, d)$	$\frac{n(t, d)}{N(d)}$
max	$TF_{max}(t, d)$	$\frac{n(t, d)}{\max_{t'} n(t', d)}$
augmented	$TF_{aug}(t, d)$	$0.5 + \frac{0.5 \cdot n(t, d)}{\max_{t'} n(t', d)}$
log	$TF_{log}(t, d)$	$\log(1 + n(t, d))$
log avg	$TF_{logavg}(t, d)$	$\frac{\log(1 + n(t, d))}{\log(1 + na(d))}$
frac	$TF_{frac}(t, d; k)$	$\frac{n(t, d)}{n(t, d) + k}$
BM25	$TF_{BM25}(t, d, c; k, b)$	$\frac{n(t, d)}{n(t, d) + k(b \cdot ndl(d, c) + (1 - b))}$

- ⊙ $|d|$: number of distinct terms in document d
- ⊙ $|c|$: number of documents in collection c
- ⊙ $n(t, d)$: number of occurrences of term t in document d
- ⊙ $N(d) = \sum_t n(t, d)$: length (overall number of occurrences of all terms) in document d
- ⊙ $na(d) = \frac{1}{|d|} \sum_t n(t, d)$: average number of occurrences of terms in document d
- ⊙ $ndl(d, c) = \frac{N(d)}{adl(c)}$: length of document d normalized wrt collection c
- ⊙ $adl(d, c) = \frac{1}{|c|} \sum_{d \in c} N(d)$: average length of documents in collection c

Variants of tf weighting

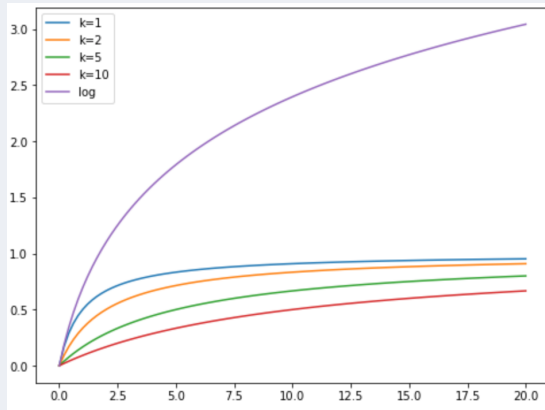
- ⊙ TF_{total} , TF_{sum} , TF_{max} all correspond to assuming “independence” of occurrences: tf increases by a same amount for each successive occurrence (independently from the number of occurrences already observed)
- ⊙ TF_{total} has no normalization wrt document length: biased toward longer documents
- ⊙ assume a set of documents of different length with the same fraction of occurrences of a certain term t : how do we want documents scored wrt t ?

Variants of tf weighting

- ⊙ TF_{total} has no normalization wrt document length: longer documents receive higher score (this could happen even for a lower fraction of occurrences, since only the absolute amount of occurrences is considered)
- ⊙ TF_{sum} normalizes wrt document length: all documents receive the same score, but perhaps we would prefer longer documents to be preferred in a certain amount, even if the fraction of term occurrences is the same
- ⊙ TF_{max} is an intermediate approach: for a same fraction of occurrences, longer documents are preferred, but not as much as in TF_{total}

Variants of tf weighting

- ⊙ TF_{frac} introduces a decreasing marginal gain wrt the number of occurrences: its increase deriving from the n -th occurrence of a term is smaller for larger n



- ⊙ the same holds for TF_{log}

Variants of idf weighting

total	$\text{idf}_{total}(t, c)$	$-\log n(t, c)$
sum	$\text{idf}_{sum}(t, c)$	$-\log \frac{n(t, c)}{ c }$
smooth sum	$\text{idf}_{smooth}(t, c)$	$-\log \frac{n(t, c) + 0.5}{ c + 1}$
prob	$\text{idf}_{prob}(t, c)$	$\max\left(0, -\log \frac{n(t, c)}{ c - n(t, c)}\right)$
smooth prob	$\text{idf}_{smoothprob}(t, c)$	$\max\left(0, -\log \frac{n(t, c) + 0.5}{ c - n(t, c) + 0.5}\right)$

- ⊙ $|c|$: number of documents in collection c
- ⊙ $n(t, c)$: number of documents in collection c in which term t occurs